НИЯУ МИФИ
Кафедра №71 «ФБИУКС»
Дисциплины: «Основы программирования», «Основы теоретической информатики»

Лабораторная работа №2

Общие требования к обоим вариантам

Тема: Разработка процедурной программы обработки данных

Варианты: 1 (Турнир), 2 (Заказы магазина)

Москва — 2026

Содержание

Цель работы 3
Процедурная декомпозиция 4
Разделение ответственностей 4
Требования к функциям 4
Разделение обработки и вывода 6
Обязательные средства Python 7
Ограничения 8
Обработка ошибок 9
Требования к входному файлу 10
Автоматические проверки 11
Для варианта 1 11
Для варианта 2 11
Структура сдаваемой работы 12
Содержание README.md 12
Требования к защите 13
Возможные вопросы на защите 13
Критерии оценивания 14
Обязательный минимум для зачёта 14

Цель работы

Цель — разработать консольную программу для чтения, проверки, обработки и сохранения структурированных данных средствами базового Python. В ходе работы применяются функции, условные операторы и циклы, строки и строковые методы, списки, словари и множества, вложенные коллекции, поиск, фильтрация, сортировка, списковые включения, чтение и запись файлов, обработка ошибок и автоматические проверки.

Процедурная декомпозиция
Студент самостоятельно разделяет программу на функции и определяет их количество, имена, параметры, возвращаемые значения, порядок вызовов и состав вспомогательных функций.

Разделение ответственностей
В программе должны быть разделены следующие ответственности:

№	Ответственность
1	Чтение данных из файла.
2	Разбор одной строки.
3	Проверка корректности записи.
4	Вычисление производных показателей.
5	Определение категории.
6	Поиск и фильтрация.
7	Сортировка.
8	Расчёт статистики.
9	Вывод данных.
10	Формирование и сохранение отчёта.
11	Обработка команд меню.

Таблица 1. Ответственности, которые должны быть разделены.

Не требуется создавать отдельную функцию для каждого пункта. Главное — понятная структура программы.

Требования к функциям
Каждая функция должна:

- решать одну понятную задачу;
- иметь содержательное имя;
- получать данные через параметры;
- возвращать результат, если он нужен другим частям программы;
- не использовать глобальные переменные без обоснованной необходимости;
- не изменять переданные коллекции, если изменение не входит в её назначение;
- содержать краткую строку документации.

Недопустимо	Допустимо
`f`	`calc_order_cost`
`func`	`load_orders`
`function1`	`filter_by_status`
`do`	`category_by_cost`
`new_func`	`validate_record`
`process_all`	`build_report`

Таблица 2. Примеры неинформативных и информативных имён.

Разделение обработки и вывода

Функции обработки данных по возможности должны возвращать результат, а не только выводить его на экран. Результат фильтрации должен быть пригоден как для вывода в консоль, так и для записи в отчёт и для автоматической проверки.

В одной функции не следует без необходимости объединять:

- запрос пользовательского ввода;
- расчёты;
- вывод;
- запись в файл.

Обязательные средства Python
В программе необходимо использовать перечисленные ниже средства.

Средство	Назначение
Список словарей	Основная структура хранения записей.
Множество (минимум одно)	Уникальные значения, проверки принадлежности.
Циклы for и while	Перебор элементов, повторяющиеся действия.
Условные операторы	Ветвления логики.
Включение коллекции (минимум одно)	Компактная фильтрация и преобразование.
sorted с параметром key	Сортировка по произвольному критерию.
Составной критерий сортировки	Несколько уровней сравнения.
Методы strip, split, lower или casefold	Очистка и нормализация текста.
with open для чтения и записи	Безопасная работа с файлами.
try и except	Обработка ожидаемых ошибок.
Форматирование строк	Красивый вывод и сборка отчётов.
Функции с параметрами и возвратом	Структурирование кода.

Таблица 3. Обязательные средства Python.

Допускается использовать стандартные функции: `len`, `sum`, `min`, `max`, `round`, `any`, `all`, `enumerate`, `sorted`.

Ограничения
В лабораторной работе не следует использовать:

- классы;
- базы данных;
- `pandas`;
- `numpy`;
- сторонние библиотеки;
- глобальные переменные для хранения основных данных;
- `except Exception` без обоснования;
- подавление ошибок через пустой `except`;
- готовые решения из библиотек, полностью заменяющие требуемую обработку.

Вся основная логика должна быть реализована средствами базового Python.

Обработка ошибок
Программа должна корректно обрабатывать следующие ситуации:

№	Ситуация
1	Отсутствие входного файла.
2	Невозможность преобразования строки в число.
3	Неправильное количество полей.
4	Пустое обязательное поле.
5	Повторяющийся идентификатор.
6	Недопустимое числовое значение.
7	Неизвестный статус (для варианта 2).
8	Неизвестный пункт меню.
9	Отсутствие результатов поиска.
10	Отсутствие корректных записей.
11	Невозможность вычислить статистический показатель для пустого списка.

Таблица 4. Ошибки, подлежащие обработке.

Не допускается конструкция `except Exception: pass`. Пользователь должен получить понятное сообщение о возникшей проблеме.

Требования к входному файлу
Входной файл должен содержать не менее 15 непустых строк, среди которых:

- не менее 10 корректных записей;
- строка с неправильным количеством полей;
- строка с неправильным числовым значением;
- строка с числом за пределами допустимого диапазона;
- строка с пустым обязательным полем;
- строка с повторяющимся идентификатором.

Для варианта 2 дополнительно должна присутствовать строка с неизвестным статусом. Студент самостоятельно готовит тестовый входной файл.

Автоматические проверки

Программа должна содержать не менее шести автоматических проверок с помощью `assert`.

Для варианта 1

- вычисление итогового балла;
- вычисление среднего балла;
- определение категории результата;
- граничные значения категорий;
- фильтрация по группе;
- поиск по части имени;
- порядок участников в рейтинге;
- условие прохождения отбора.

Для варианта 2

- вычисление стоимости заказа;
- определение категории стоимости;
- граничные значения категорий;
- фильтрация по статусу;
- фильтрация по категории;
- поиск по части имени покупателя;
- порядок сортировки;
- расчёт выручки только по выполненным заказам.

Студент сам выбирает проверяемые функции, тестовые данные и ожидаемые результаты. Проверки должны включать обычные и граничные случаи.

Структура сдаваемой работы

```
lab_02/
  main.py
  input.txt
  report.txt
  README.md
```

Допускается использовать предметные имена файлов:

- `participants.txt` и `tournament_report.txt` — для варианта 1;
- `orders.txt` и `orders_report.txt` — для варианта 2.

Разделение программы на несколько файлов Python не является обязательным.

Содержание README.md

Файл `README.md` должен содержать:

- ФИО студента;
- учебную группу;
- номер варианта;
- краткое описание задачи;
- формат входного файла;
- инструкцию по запуску;
- описание структуры программы;
- перечень основных возможностей;
- примеры корректной и некорректной записи;
- перечень обработанных ошибок.

Требования к защите
На защите студент должен:

№	Пункт демонстрации
1	Запустить программу на подготовленном файле.
2	Продемонстрировать обработку корректных и некорректных записей.
3	Показать поиск, фильтрацию и сортировку.
4	Продемонстрировать формирование отчёта.
5	Объяснить структуру используемых коллекций.
6	Объяснить декомпозицию программы на функции.
7	Указать параметры и возвращаемые значения выбранной функции.
8	Объяснить, изменяют ли функции исходные списки.
9	Показать автоматические проверки.
10	Внести небольшое изменение в программу по заданию преподавателя.

Таблица 5. Что требуется на защите.

Возможные вопросы на защите

- Почему для хранения записей выбран список словарей?
- Для чего в программе используется множество?
- Чем сортировка отличается от фильтрации?
- Почему присваивание одного списка другому не создаёт копию?
- Как сохранить исходный порядок записей при построении рейтинга?
- Чем `return` отличается от `print`?
- Почему обработка исключений не заменяет проверку данных?
- Что произойдёт при обработке пустого файла?

Критерии оценивания

Работа зачтена, если получены 6 баллов (с учётом корректных ответов на вопросы).

Критерий	Баллы
Чтение и разбор входного файла	1
Проверка записей и обработка ошибок	1,5
Корректное применение коллекций	1
Декомпозиция программы на функции	1,5
Поиск и фильтрация	1
Сортировка по нескольким критериям	1
Расчёт статистики	1
Формирование и сохранение отчёта	0,5
Автоматические проверки и граничные случаи	1
Качество кода и защита	0,5
Итого	**10**

Таблица 6. Критерии оценивания.

Обязательный минимум для зачёта

- чтение данных из файла;
- хранение данных в списке словарей;
- процедурная декомпозиция;
- проверка входных данных;
- обработка отсутствующего файла;
- поиск или фильтрация;
- сортировка;
- расчёт статистики;
- запись отчёта;
- минимум шесть автоматических проверок;
- объяснение собственного решения на защите.